

INVOICE PORTAL

Complete Project Notes

Laravel 11 | PHP 8.3 | MySQL | Tailwind CSS | Stripe | Railway

Nehal Khan | June 2026

Section 1: Project Overview

Invoice Portal is a SaaS-based Invoice Management System. Business owners can manage clients, create invoices, accept Stripe payments, send emails, and give clients their own portal access.

1.1 Live URLs

- Live Site: <https://web-production-15ad2e.up.railway.app>
- GitHub: https://github.com/nehaal-dev/invoice_portal

1.2 Tech Stack

Technology	Purpose
Laravel 11	Backend MVC Framework
PHP 8.3	Server-side Language
MySQL	Database
Tailwind CSS	Frontend UI Styling
Laravel Breeze	Authentication (session-based)
Laravel Sanctum	API Authentication (token-based)
barryvdh/laravel-dompdf	PDF Generation
Stripe PHP SDK	Payment Gateway
Gmail SMTP	Email Sending
Railway.app	Deployment Platform

1.3 Complete Features

- User Authentication (Register/Login/Logout)
- Client CRUD (Create, Read, Update, Delete)
- Invoice CRUD with multiple line items
- Automatic invoice number generation (INV-001, INV-002...)
- PDF Download (barryvdh/laravel-dompdf)
- Stripe Payment Integration (Test Mode)
- Email Notifications with PDF attachment (Gmail SMTP)
- Dashboard with business statistics
- REST API with Sanctum Bearer token authentication
- Subscription Plans (Free: 5 invoices/month, Pro: unlimited)
- Client Portal (clients view their own invoices)
- Deployment on Railway.app with MySQL

Section 2: Laravel Basics

2.1 What is Laravel?

Laravel is a PHP web application framework that follows the MVC (Model-View-Controller) pattern. It provides tools like routing, ORM (Eloquent), authentication, middleware, and more out of the box.

2.2 MVC Pattern

Component	Role
Model	Represents database table. Contains relationships and business logic. Example: Invoice.php
View	What user sees — HTML/Blade templates. Example: invoices/index.blade.php
Controller	Bridge between Model and View. Handles requests and returns responses. Example: InvoiceController.php

2.3 Laravel Request Lifecycle

- User sends HTTP request (e.g. GET /invoices)
- Request goes to public/index.php (entry point)
- Laravel boots — loads service providers
- Request passes through Middleware (auth check, CSRF, etc.)
- Router matches URL to Controller method
- Controller calls Model (database query)
- Controller returns View (HTML) or JSON
- Response sent back to user

2.4 Important Artisan Commands

```
php artisan serve // Start local server
php artisan make:model ModelName -m // Create Model + Migration
php artisan make:controller ControllerName // Create Controller
php artisan make:migration migration_name // Create Migration
php artisan make:middleware MiddlewareName // Create Middleware
php artisan make:mail MailClassName // Create Mailable
php artisan migrate // Run all pending migrations
php artisan migrate:rollback // Undo last migration
php artisan route:list // Show all routes
php artisan tinker // Interactive PHP shell
php artisan config:clear // Clear config cache
php artisan cache:clear // Clear application cache
```

2.5 Folder Structure

Folder/File	Purpose
app/Models/	Eloquent Models (User.php, Invoice.php, Client.php)
app/Http/Controllers/	Controllers (InvoiceController.php etc.)
app/Http/Middleware/	Middleware files
app/Mail/	Mailable classes for emails
database/migrations/	Database schema files
resources/views/	Blade template files
routes/web.php	Web routes (browser requests)
routes/api.php	API routes (JSON requests)
.env	Environment variables (DB, mail, Stripe keys)
bootstrap/app.php	App configuration, middleware registration

Section 3: Database — Migrations & Models

3.1 What is a Migration?

A Migration is a PHP file that defines the structure of a database table. Instead of manually creating tables in phpMyAdmin, we write migrations in code — so the database schema is version controlled.

```
php artisan make:migration create_clients_table
```

Migration file structure:

```
public function up(): void {
    Schema::create('clients', function (Blueprint $table) {
        $table->id(); // auto-increment primary key
        $table->foreignId('user_id'); // foreign key
        $table->string('client_name'); // VARCHAR column
        $table->string('email')->nullable();
        $table->decimal('total', 10, 2); // decimal number
        $table->timestamps(); // created_at, updated_at
    });
}
```

NOTE: Always run `php artisan migrate` after creating migration files.

3.2 Project Database Schema

Table	Key Columns
users	id, name, email, password, role (owner/client), client_id
clients	id, user_id, client_name, email, phone, address, city, country
invoices	id, user_id, client_id, invoice_number, invoice_date, due_date, status, subtotal, tax, discount, total, notes
items	id, invoice_id, item_name, quantity, price, total
payments	id, invoice_id, amount, payment_date, payment_method, transaction_id
subscriptions	id, user_id, plan_name, status, amount, starts_at, expires_at
personal_access_tokens	id, tokenable_type, tokenable_id, name, token (Sanctum API tokens)

3.3 Eloquent Models

A Model is a PHP class that represents a database table. One row in the table = one Model object.

```
class Invoice extends Model {
    protected $fillable = ['user_id', 'client_id', 'invoice_number', ...];
    // Relationships
    public function client() { return $this->belongsTo(Client::class); }
    public function items() { return $this->hasMany(InvoiceItem::class); }
    public function payment() { return $this->hasOne(Payment::class); }
}
```

```
}
```

NOTE: InvoiceItem model uses protected \$table = 'items' because table name is 'items' not 'invoice_items'.

3.4 Model Relationships

Relationship	Example
hasMany	User hasMany Invoices — one user has many invoices
belongsTo	Invoice belongsTo Client — each invoice belongs to one client
hasOne	Invoice hasOne Payment — each invoice has one payment
hasMany	Invoice hasMany Items — one invoice has many line items

3.5 Eloquent Common Methods

```
// Fetch all records
Invoice::all();

// Filter by column
Invoice::where('user_id', auth()->id()->get());

// Find by ID
Invoice::find(1);

// Create new record
Invoice::create(['user_id' => 1, 'status' => 'draft']);

// Update
$invoice->update(['status' => 'paid']);

// Delete
$invoice->delete();
Invoice::destroy($id);
```

Section 4: Routing

4.1 What is a Route?

A Route maps an HTTP request (URL + method) to a Controller method. All web routes are in `routes/web.php` and API routes are in `routes/api.php`.

4.2 Route Syntax

```
Route::get('/invoices', [InvoiceController::class, 'index']);
Route::post('/invoices', [InvoiceController::class, 'store']);
Route::get('/invoices/{invoice}', [InvoiceController::class, 'show']);
Route::patch('/invoices/{invoice}', [InvoiceController::class, 'update']);
Route::delete('/invoices/{invoice}', [InvoiceController::class, 'destroy']);
```

4.3 Route Names

```
Route::get('/invoices', [InvoiceController::class, 'index'])->
    name('invoices.index');
// Use in Blade: {{ route('invoices.index') }}
// Use in Controller: redirect()->route('invoices.index')
```

4.4 Route Model Binding

When you use `{invoice}` in route and `Invoice $invoice` in controller, Laravel automatically fetches the Invoice record from database using the ID in URL.

```
// Route
Route::get('/invoices/{invoice}', [InvoiceController::class, 'show']);

// Controller - $invoice is already fetched from DB
public function show(Invoice $invoice) {
    return view('invoices.show', compact('invoice'));
}
```

4.5 Protected Routes (Middleware)

```
Route::middleware('auth')->group(function () {
    Route::resource('clients', ClientController::class);
    Route::resource('invoices', InvoiceController::class);
});
```

NOTE: All routes inside this group require user to be logged in.

Section 5: Controllers

5.1 What is a Controller?

A Controller handles incoming HTTP requests, interacts with Models (database), and returns responses (views or JSON). Each public method in a controller handles one route.

5.2 Standard 7 Methods (RESTful)

Method	Action
index()	Show list of all records — GET /invoices
create()	Show form to create new record — GET /invoices/create
store(Request)	Save new record to DB — POST /invoices
show(Model)	Show one record — GET /invoices/{id}
edit(Model)	Show edit form — GET /invoices/{id}/edit
update(Request, Model)	Update record in DB — PATCH /invoices/{id}
destroy(Model)	Delete record — DELETE /invoices/{id}

5.3 InvoiceController — store() Method

Creating invoice saves to 2 tables: invoices + items.

```
public function store(Request $request) {
    // Step 1: Calculate subtotal
    $sub_total = 0;
    foreach ($request->item_name as $index => $name) {
        $sub_total += $request->quantity[$index] * $request->price[$index];
    }
    // Step 2: Calculate total
    $total = $sub_total + ($sub_total * $request->tax / 100) - $request->discount;
    // Step 3: Save invoice
    $invoice = Invoice::create([
        'user_id' => auth()->id(),
        'invoice_number' => 'INV-' . str_pad(Invoice::max('id') + 1, 3, '0',
STR_PAD_LEFT),
        'subtotal' => $sub_total,
        'total' => $total, ...
    ]);
    // Step 4: Save each item
    foreach ($request->item_name as $index => $item_name) {
        InvoiceItem::create(['invoice_id' => $invoice->id, ...]);
    }
    return redirect()->route('invoices.index');
}
```


5.4 auth() Helper

```
auth()->id()      // Get logged-in user's ID  
auth()->user()    // Get logged-in user's full object  
auth()->attempt() // Check email/password (used in API login)
```

Section 6: Blade Views

6.1 What is Blade?

Blade is Laravel's templating engine. Files end with `.blade.php`. It allows PHP code inside HTML using special syntax.

6.2 Common Blade Syntax

```
{{-- Comment --}}
{{ $variable }}           // Echo variable (XSS safe)
{!! $html !!}             // Echo raw HTML
@if($condition) ... @endif
@foreach($items as $item) ... @endforeach
@extends('layouts.app')    // Extend layout
@section('content') ... @endsection
@yield('content')         // Placeholder in layout
@csrf                    // CSRF token (required in forms)
@method('PATCH')        // Spoof HTTP method for forms
```

6.3 Form Structure

```
<form action="{{ route('invoices.store') }}" method="POST">
    @csrf
    <input type="text" name="client_name">
    <button type="submit">Save</button>
</form>
```

```
// For PATCH/DELETE:
<form action="{{ route('invoices.update', $invoice->id) }}" method="POST">
    @csrf
    @method('PATCH')
    ...
</form>
```

6.4 Displaying Data in Table

```
@foreach($invoices as $invoice)
    <tr>
        <td>{{ $invoice->invoice_number }}</td>
        <td>{{ $invoice->client->client_name }}</td>
        <td>Rs. {{ number_format($invoice->total, 2) }}</td>
    </tr>
@endforeach
```

6.5 Status Badge Pattern

```
<span class="
    {{ $invoice->status == 'paid'      ? 'bg-green-100 text-green-700' : '' }}>
```

```
    {{ $invoice->status == 'unpaid' ? 'bg-red-100 text-red-700' : '' }}
    {{ $invoice->status == 'overdue' ? 'bg-orange-100 text-orange-700' : '' }}
">
    {{ ucfirst($invoice->status) }}
</span>
```

Section 7: Authentication

7.1 Laravel Breeze

Laravel Breeze is an authentication starter kit. It provides Register, Login, Logout, Password Reset pages out of the box using session-based authentication.

```
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install && npm run dev
php artisan migrate
```

7.2 How Session Auth Works

- User submits login form with email + password
- Laravel checks credentials against users table
- If correct — creates session, stores user ID in session
- Every subsequent request — Laravel reads session, knows who is logged in
- `auth()->user()` and `auth()->id()` are available everywhere

7.3 Protecting Routes

```
// Single route
Route::get('/dashboard', ...)->middleware('auth');

// Group of routes
Route::middleware('auth')->group(function () {
    Route::resource('invoices', InvoiceController::class);
});
```

7.4 User Model — HasApiTokens

For REST API (Sanctum), User model must have HasApiTokens trait:

```
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable {
    use HasApiTokens, HasFactory, Notifiable;
}
```

NOTE: Without HasApiTokens, createToken() method will not work — 500 error.

Section 8: Client CRUD

8.1 Flow

- Route receives request
- Controller method runs
- Model interacts with database
- Controller returns view or redirect

8.2 Client Routes

```
Route::resource('clients', ClientController::class);
```

This single line creates all 7 routes (index, create, store, show, edit, update, destroy).

8.3 ClientController — store()

```
public function store(Request $req) {
    $req->validate([
        'client_name' => 'required|string|max:200',
        'email' => 'nullable|email',
    ]);

    $client = Client::create([
        'user_id'      => auth()->id(),
        'client_name' => $req->client_name,
        'email'        => $req->email,
        'phone'        => $req->phone,
        'address'      => $req->client_address,
        'city'         => $req->city,
        'country'      => $req->country
    ]);

    // Create portal user for client
    if ($req->email) {
        User::create([
            'name'      => $req->client_name,
            'email'     => $req->email,
            'password'  => bcrypt('client123'),
            'role'      => 'client',
            'client_id' => $client->id,
        ]);
    }

    return redirect()->route('clients.index');
}
```

NOTE: `$client = Client::create([...])` — save return value to variable first, then use `$client->id` for portal user.

Section 9: Invoice CRUD

9.1 Invoice Create — 2 Tables

When creating an invoice, we save to 2 tables: invoices table (parent) and items table (children).

```
// Step 1: Save to invoices table
$invoice = Invoice::create([...]);

// Step 2: Save each item (uses $invoice->id as foreign key)
foreach ($request->item_name as $index => $item_name) {
    InvoiceItem::create([
        'invoice_id' => $invoice->id,
        'item_name'   => $item_name,
        'quantity'    => $request->quantity[$index],
        'price'       => $request->price[$index],
        'total'       => $request->quantity[$index] * $request->price[$index]
    ]);
}
```

9.2 Invoice Number Generation

```
'invoice_number' => 'INV-' . str_pad(Invoice::max('id') + 1, 3, '0', STR_PAD_LEFT)
// Invoice::max('id') = highest ID in table
// +1 = next number
// str_pad = pad with zeros to 3 digits: 1 -> 001
```

9.3 Tax & Discount Calculation

```
// Tax is PERCENTAGE
// Discount is FLAT AMOUNT (Rs.)
$sub_total = sum of (quantity * price) for all items
$total = $sub_total + ($sub_total * $tax / 100) - $discount

// Example: subtotal=100, tax=18%, discount=Rs.10
// total = 100 + 18 - 10 = 108
```

9.4 Dynamic Items (JavaScript)

```
// Add Item button in create form
document.getElementById('add-items').addEventListener('click', function() {
    var newRow = document.createElement('div');
    newRow.innerHTML = `
        <input type="text" name="item_name[]">
        <input type="number" name="quantity[]">
        <input type="number" name="price[]">
    `;
    container.appendChild(newRow);
});
```

NOTE: name='item_name[]' — square brackets create PHP array in \$request.

9.5 Invoice Update

```
public function update(Request $request, Invoice $invoice) {
    $invoice->update([...]);          // Update invoice
    $invoice->items()->delete();       // Delete old items
    foreach ($request->item_name ...) { // Add new items
        InvoiceItem::create([...]);
    }
    return redirect()->route('invoices.index');
}
```

Section 10: PDF Generation

10.1 Package

```
composer require barryvdh/laravel-dompdf
```

10.2 How DomPDF Works

- Load a Blade view into DomPDF
- DomPDF converts HTML+CSS to PDF
- Only inline CSS works in PDF (no Tailwind)
- Return as download or stream

10.3 Controller Code

```
use Barryvdh\DomPDF\Facade\Pdf;

public function downloadPdf(Invoice $invoice) {
    $items = InvoiceItem::where('invoice_id', $invoice->id)->get();
    $clint = Client::find($invoice->client_id);
    $pdf = Pdf::loadView('invoices.pdf', compact('invoice', 'items', 'clint'));
    return $pdf->download('Invoice-' . $invoice->invoice_number . '.pdf');
}
```

NOTE: Use \$pdf->stream() to preview in browser, \$pdf->download() to force download.

10.4 Production Config

In config/dompdf.php — enable_remote must be true for production:

```
'enable_remote' => true,
```

NOTE: Without this, PDF generation fails on production server.

Section 11: Email Notifications

11.1 Gmail SMTP Setup

.env configuration:

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=your_gmail@gmail.com
MAIL_PASSWORD=your_app_password_16chars
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=your_gmail@gmail.com
MAIL_FROM_NAME="Invoice Portal"
```

NOTE: Use Gmail App Password (not regular password). Enable 2-Step Verification first, then generate App Password.

11.2 What is a Mailable Class?

A Mailable class is Laravel's email blueprint — it defines the subject, view (template), and attachments for an email.

```
php artisan make:mail InvoiceMail
```

11.3 InvoiceMail Class

```
class InvoiceMail extends Mailable {
    use Queueable, SerializesModels;
    public Invoice $invoice;

    public function __construct(Invoice $invoice) {
        $this->invoice = $invoice; // No return here!
    }

    public function envelope(): Envelope {
        return new Envelope(
            subject: 'Invoice ' . $this->invoice->invoice_number,
        );
    }

    public function content(): Content {
        return new Content(view: 'emails.invoice');
    }

    public function attachments(): array {
        $pdf = Pdf::loadView('invoices.pdf', [
            'invoice' => $this->invoice,
            'items'   => $this->invoice->items,
            'clint'   => $this->invoice->client,
        ]);
        return [

```

```
        Attachment::fromData(  
            fn() => $pdf->output(),  
            'Invoice-' . $this->invoice->invoice_number . '.pdf'  
        )->withMime('application/pdf'),  
    ];  
}  
}
```

11.4 Sending Email from Controller

```
use Illuminate\Support\Facades\Mail;  
use App\Mail\InvoiceMail;  
  
public function sendEmail(Invoice $invoice) {  
    Mail::to($invoice->client->email)->send(new InvoiceMail($invoice));  
    return redirect()->route('invoices.show', $invoice->id)  
        ->with('success', 'Email sent!');  
}
```

Section 12: Stripe Payment Integration

12.1 Setup

```
composer require stripe/stripe-php
```

.env:

```
STRIPE_KEY=pk_test_...  
STRIPE_SECRET=sk_test_...
```

12.2 Payment Flow

- User clicks Pay Now on invoice page
- checkout() method creates Stripe Session
- User redirected to Stripe's payment page
- User enters test card: 4242 4242 4242 4242
- Stripe redirects to success_url
- paymentSuccess() saves Payment record
- Invoice status updated to 'paid'
- Payment confirmation email sent

12.3 checkout() Method

```
public function checkout Invoice $invoice) {  
    Stripe::setApiKey(env('STRIPE_SECRET'));  
    $session = Session::create([  
        'payment_method_types' => ['card'],  
        'line_items' => [[  
            'price_data' => [  
                'currency' => 'inr',  
                'product_data' => ['name' => 'Invoice ' . $invoice->invoice_number],  
                'unit_amount' => $invoice->total * 100, // Stripe uses paise  
            ],  
            'quantity' => 1,  
        ]],  
        'mode' => 'payment',  
        'success_url' => route('invoices.payment.success', $invoice->id) . '?session_id={CHECKOUT_SESSION_ID}',  
        'cancel_url' => route('invoices.show', $invoice->id),  
    ]);  
    return redirect($session->url);  
}
```

12.4 paymentSuccess() Method

```
public function paymentSuccess Invoice $invoice, Request $request) {  
    Stripe::setApiKey(env('STRIPE_SECRET'));  
    $session = Session::retrieve($request->session_id);
```

```
    if ($session->payment_status === 'paid') {
        // Prevent duplicate payments
        $existing = Payment::where('transaction_id', $session->payment_intent)->first();
        if (!$existing) {
            Payment::create([...]);
            $invoice->update(['status' => 'paid']);
            Mail::to($invoice->client->email)->send(new PaymentConfirmationMail($invoice));
        }
    }
    return redirect()->route('invoices.show', $invoice->id);
}
```

NOTE: unit_amount is in smallest currency unit — for INR it is paise. Rs. 1000 = 100000 paise.

Section 13: REST API

13.1 What is REST API?

REST API allows other applications (mobile apps, third-party systems) to interact with your Laravel app using JSON instead of HTML. Browser uses web routes; mobile apps use API routes.

Type	Detail
Web Request	Browser sends request -> Laravel returns HTML page
API Request	Mobile app/Postman sends request -> Laravel returns JSON
Auth (Web)	Session cookie (Laravel Breeze)
Auth (API)	Bearer token (Laravel Sanctum)

13.2 Setup

```
php artisan install:api
```

This creates routes/api.php and personal_access_tokens table.

13.3 API Routes

```
// routes/api.php
Route::post('/login', function(Request $request) {
    if (!auth()->attempt($request->only('email', 'password'))) {
        return response()->json(['message' => 'Invalid credentials'], 401);
    }
    $token = auth()->user()->createToken('api-token')->plainTextToken;
    return response()->json(['token' => $token]);
});

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/invoices', [InvoiceApiController::class, 'index']);
    Route::get('/invoices/{invoice}', [InvoiceApiController::class, 'show']);
    Route::post('/invoices', [InvoiceApiController::class, 'store']);
    Route::get('/clients', [ClientApiController::class, 'index']);
    Route::post('/clients', [ClientApiController::class, 'store']);
});
```

13.4 API Controller

```
// app/Http/Controllers/Api/InvoiceApiController.php
namespace App\Http\Controllers\Api; // UPPERCASE Api

public function index() {
    $invoices = Invoice::where('user_id', auth()->id()->get());
    return response()->json($invoices);
}
```

```
public function show Invoice $invoice) {  
    return response()->json($invoice);  
}  
  
public function store(Request $request) {  
    $invoice = Invoice::create([...]);  
    return response()->json(['message' => 'Created', 'invoice' => $invoice], 201);  
}
```

13.5 Testing with Postman

- Step 1: POST /api/login with email and password -> get token
- Step 2: GET /api/invoices with Authorization: Bearer TOKEN
- Step 3: Always add Accept: application/json header

NOTE: Api folder name must be uppercase 'Api' not 'api' — case sensitivity causes 500 error on Linux servers.

NOTE: bootstrap/app.php must have \$middleware->statefulApi() to fix CSRF issues on API.

Section 14: Subscription System

14.1 How it Works

- Free plan: max 5 invoices per month
- Pro plan: unlimited invoices (Rs. 999/month)
- CheckSubscription middleware runs before invoice create
- If free user has 5+ invoices this month, redirect to subscription page
- User can upgrade via Stripe checkout

14.2 CheckSubscription Middleware

```
public function handle(Request $request, Closure $next) {
    $subscription = Subscription::where('user_id', auth()->id()->first();
    $isPro = $subscription && $subscription->plan_name === 'pro'
        && $subscription->status === 'active';

    if (!$isPro) {
        $count = Invoice::where('user_id', auth()->id())
            ->whereMonth('created_at', now()->month)->count();
        if ($count >= 5) {
            return redirect()->route('subscription.index')
                ->with('error', 'Free plan limit reached. Upgrade to Pro.');
        }
    }
    return $next($request);
}
```

14.3 Registering Middleware

In bootstrap/app.php:

```
->withMiddleware(function (Middleware $middleware): void {
    $middleware->statefulApi();
    $middleware->alias([
        'check.subscription' => \App\Http\Middleware\CheckSubscription::class,
        'client' => \App\Http\Middleware\ClientMiddleware::class,
    ]);
})
```

Section 15: Client Portal

15.1 Concept

Client portal allows clients to login and view their own invoices. A separate 'client' role is added to users table. Login page is shared but redirects differ based on role.

User Type	After Login
owner	Redirects to /dashboard
client	Redirects to /portal

15.2 Users Table Changes

```
// Migration
$table->enum('role', ['owner', 'client'])->default('owner');
$table->integer('client_id')->nullable();
```

15.3 Login Redirect Logic

```
// AuthenticatedSessionController.php
public function store(LoginRequest $request) {
    $request->authenticate();
    $request->session()->regenerate();

    if (auth()->user()->role === 'client') {
        return redirect()->route('portal.index');
    }
    return redirect()->intended(route('dashboard'));
}
```

15.4 Client Portal Controller

```
public function index() {
    $invoices = Invoice::where('client_id', auth()->user()->client_id)->get();
    return view('client-portal.index', compact('invoices'));
}

public function show(Invoice $invoice) {
    if ($invoice->client_id !== auth()->user()->client_id) {
        abort(403); // Forbidden - can't see other client's invoice
    }
    return view('client-portal.show', compact('invoice', 'items'));
}
```

15.5 ClientMiddleware

```
public function handle(Request $request, Closure $next) {
```



```
if (auth()->user()->role !== 'client') {  
  return redirect()->route('dashboard');  
}  
return $next($request);  
}
```

Section 16: Deployment on Railway.app

16.1 Steps

- Create Railway.app account (free trial available)
- New Project -> Deploy from GitHub repo
- Add MySQL database service
- Set environment variables
- Set Start Command
- Deploy

16.2 Required Environment Variables

```
APP_NAME=Invoice Portal
APP_ENV=production
APP_KEY=base64:... (from local .env)
APP_DEBUG=false
APP_URL=https://your-app.up.railway.app
ASSET_URL=https://your-app.up.railway.app
DB_CONNECTION=mysql
DB_HOST=${{MySQL.MYSQLHOST}}
DB_PORT=3306
DB_DATABASE=${{MySQL.MYSQLDATABASE}}
DB_USERNAME=${{MySQL.MYSQLUSER}}
DB_PASSWORD=${{MySQL.MYSQLPASSWORD}}
STRIPE_KEY=pk_test...
STRIPE_SECRET=sk_test...
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=your@gmail.com
MAIL_PASSWORD=app_password
MAIL_ENCRYPTION=tls
SESSION_DRIVER=database
```

16.3 Start Command

```
php artisan migrate --force && php artisan storage:link && php artisan serve --
host=0.0.0.0 --port=$PORT
```

16.4 Common Production Fixes Applied

Issue	Fix
CSS not loading	Remove /public/build from .gitignore, run npm run build, push
Mixed Content HTTPS error	AppServiceProvider: URL::forceScheme('https')
PDF download failing	config/dompdf.php: enable_remote => true

Issue	Fix
API folder case	git mv api Api_temp then git mv Api_temp Api
personal_access_tokens exists	Add Schema::hasTable() check in migration
CSRF on API routes	bootstrap/app.php: \$middleware->statefulApi()

Section 17: Mistakes Made During Practice

17.1 Controller Mistakes

Mistake	Correct Code
Used Controller name instead of Model: InvoiceApiController::where(...)	Invoice::where('user_id', auth()->id())->get()
Used return in constructor: return \$this->invoice = \$invoice	\$this->invoice = \$invoice (no return)
Forgot to save Client::create() to variable, then used \$client->id	\$client = Client::create([...]); then \$client->id
Used lowercase namespace: App\Http\Controllers\api	App\Http\Controllers\Api (uppercase A)

17.2 Blade Mistakes

Mistake	Fix
Nested nav links inside Dashboard nav link	Keep all nav links as siblings, not nested
Used HTTP method PUT in form without @method directive	Add @method('PATCH') inside form
Forgot @csrf in form	Always add @csrf as first line inside form

17.3 Git Mistakes

Mistake	Fix
Pushed API folder as lowercase 'api', Linux treats differently	git mv api Api_temp then git mv Api_temp Api
Forgot to remove /public/build from .gitignore	Edit .gitignore, remove that line, push build files
Diverged branches after pull	git pull origin main --no-rebase

17.4 Deployment Mistakes

Mistake	Fix
DB_PORT used \${MySQL.MYSQLPORT} reference (failed)	Set DB_PORT=3306 as plain value
MYSQLPASSWORD used instead of DB_PASSWORD	Rename variable to DB_PASSWORD
personal_access_tokens migration failed (table exists)	Add Schema::hasTable() check in migration up()
User model missing HasApiTokens	Add use HasApiTokens in User model

Section 18: Doubts Asked During Development

Q1: LEFT part mein table column naam, RIGHT part mein kya hota hai?

RIGHT part = jo naam form mein (name attribute) ya API JSON mein aata hai. Tu khud decide karta hai. Mostly dono same hote hain.

```
'client_name' => $request->client_name // left=DB column, right=form field name
```

Q2: API mein form nahi hota to data kaise aata hai?

API mein Postman ya mobile app JSON bhejta hai. `$request->fieldname` se JSON ka data milta hai, bilkul form jaisa. Fark sirf source ka hai.

Q3: Request class web aur API dono mein kyun use hoti hai?

Request class form data aur JSON data dono handle karti hai. Laravel automatically detect karta hai. Fark sirf response mein hota hai — web mein HTML return, API mein JSON return.

Q4: `$request->status ?? 'draft'` kya karta hai?

`??` is null coalescing operator. Agar `$request->status` null hai (not sent) to 'draft' use karo. Agar value hai to woh use karo.

Q5: `response()->json(['message' => '...'], 201)` mein 201 kya hai?

201 = HTTP status code for 'Created'. 200 = OK, 201 = Created, 401 = Unauthorized, 404 = Not Found, 500 = Server Error.

Q6: `auth()->attempt()`, `auth()->user()`, `createToken()` kahan se aate hain?

`auth()->attempt()` and `auth()->user()` = Laravel built-in Auth system. `createToken()` = Laravel Sanctum package. Yeh sab built-in methods hain, khud nahi banaye.

Q7: Discount flat amount hai ya percentage?

Tera project mein flat amount (Rs.) hai. Tax percentage hai. Code: $\text{total} = \text{subtotal} + (\text{subtotal} * \text{tax}/100) - \text{discount}$

Q8: Middleware kya hota hai?

Middleware = request aane se pehle jo check hota hai. Auth middleware check karta hai user logged in hai ya nahi. CheckSubscription middleware check karta hai invoice limit cross hua ya nahi.

Q9: API folder lowercase aur uppercase issue kyun?

Mac ka file system case-insensitive hai isliye local pe 'api' aur 'Api' same dikh rahe the. Linux (Railway server) case-sensitive hai. Production pe App\Http\Controllers\Api match nahi ho raha tha lowercase folder se.

Q10: Pehle features banana chahiye ya deploy?

Pehle deploy karna best practice hai — live URL milta hai resume ke liye. Baad mein features add karo, git push karo, Railway automatically redeploy karta hai.

Section 19: Top 30 Interview Questions

Laravel Basics

Q1: Laravel kya hai aur kyun use karte hain?

Laravel PHP ka MVC framework hai. Routing, ORM, auth, middleware, email sab built-in milta hai. Repetitive kaam avoid hota hai, code clean hota hai.

Q2: MVC pattern explain karo.

Model = database logic. View = UI (Blade templates). Controller = bridge between model and view. Request aaya, Controller ne Model se data liya, View ko diya.

Q3: Route Model Binding kya hota hai?

Jab route mein {invoice} likhte hain aur controller mein Invoice \$invoice likhte hain, Laravel automatically DB se record fetch karta hai. Manual Invoice::find(\$id) ki zaroorat nahi.

Q4: Middleware kya hai? Project mein kaunse middleware use kiye?

Middleware request ke beech mein filter ka kaam karta hai. Project mein: auth (login check), check.subscription (invoice limit), client (portal access). bootstrap/app.php mein register karte hain.

Q5: Eloquent ORM kya hai?

Eloquent = Laravel ka Object-Relational Mapper. Database tables ko PHP classes se map karta hai. Invoice::where()->get() jaisi readable queries likhte hain instead of raw SQL.

Q6: hasMany aur belongsTo relationships explain karo.

hasMany = ek record ke multiple child records. User hasMany Invoices. belongsTo = child ka parent. Invoice belongsTo Client. Foreign key belongsTo wale model mein hoti hai.

Project Specific

Q7: Invoice create karte waqt 2 tables mein kaise save kiya?

Pehle Invoice::create() se invoices table mein save kiya aur \$invoice->id mila. Phir foreach loop mein har item ke liye InvoiceItem::create() se items table mein save kiya using \$invoice->id as foreign key.

Q8: Invoice number auto-generate kaise kiya?

```
'INV-' . str_pad(Invoice::max('id') + 1, 3, '0', STR_PAD_LEFT)
```

Invoice::max('id') = highest ID. +1 = next number. str_pad = 3 digits with leading zeros.

Q9: Tax aur Discount calculation kaise ki?

Tax = percentage of subtotal. Discount = flat amount (Rs.). Formula: total = subtotal + (subtotal * tax/100) - discount

Q10: PDF generation kaise implement kiya?

barryvdh/laravel-dompdf package use kiya. Blade view banaya with inline CSS only (no Tailwind in PDF). Pdf::loadView() se PDF banaya. download() se force download, stream() se browser preview.

Q11: Stripe payment integration explain karo.

checkout() method Stripe Session create karta hai with invoice amount (paise mein). User Stripe page pe redirect hota hai. Payment ke baad paymentSuccess() mein session retrieve karte hain, status check karte hain, Payment record save karte hain, invoice status paid karte hain. Duplicate prevention ke liye transaction_id check karte hain.

Q12: Email kaise bheja? PDF attachment kaise add kiya?

Gmail SMTP configure kiya .env mein. InvoiceMail Mailable class banai. attachments() method mein DomPDF se PDF generate kiya aur Attachment::fromData() se attach kiya. Mail::to()->send() se bheja.

REST API

Q13: REST API kya hai aur project mein kyun banaya?

REST API allows other apps (mobile) to interact with Laravel using JSON instead of HTML. Project mein banaya taaki future mein mobile app ya third-party integration ho sake. Interview mein API puchhte hain.

Q14: Sanctum kya hai? Web auth se kaise alag hai?

Sanctum = API authentication. Web auth session cookies use karta hai (stateful). Sanctum Bearer tokens use karta hai (stateless). Har API request mein Authorization: Bearer TOKEN header bhejte hain.

Q15: POST /api/login ka flow explain karo.

Credentials check kiye auth()->attempt() se. Success pe createToken() se Sanctum token banaya. plainTextToken se string extract kiya. JSON mein token return kiya. Client is token ko store karta hai aur future requests mein bhejta hai.

Q16: API aur Web controller mein kya fark hai?

Web controller HTML/view return karta hai. API controller JSON return karta hai. Business logic same reh sakta hai, sirf response format alag hota hai.

Deployment

Q17: Railway.app pe deploy kaise kiya?

GitHub se Railway ko connect kiya. MySQL service add ki. Environment variables set kiye (APP_KEY, DB_*, STRIPE_*, MAIL_*). Start command: php artisan migrate --force && php artisan storage:link && php artisan serve.

Q18: Production pe CSS kyu nahi dikh raha tha? Kaise fix kiya?

/public/build .gitignore mein tha. npm run build se assets compile kiye. .gitignore se /public/build line remove ki. Build files push kiye. AppServiceProvider mein URL::forceScheme('https') add kiya Mixed Content fix ke liye.

Q19: API folder case issue kya tha? Kaise fix kiya?

Mac case-insensitive hai, Linux case-sensitive. Local pe 'api' aur 'Api' same tha lekin production pe nahi. git mv api Api_temp phir git mv Api_temp Api se rename kiya.

General PHP/Laravel

Q20: \$fillable kya karta hai?

\$fillable = whitelist of columns that can be mass-assigned. Model::create([...]) sirf \$fillable mein likhe columns hi save karta hai. Security ke liye — prevents mass assignment vulnerability.

Q21: compact() function kya karta hai?

compact('invoice', 'items') = ['invoice' => \$invoice, 'items' => \$items]. Variable names se automatically array banata hai. View ko data pass karne ke liye use karte hain.

Q22: redirect()->route() aur redirect()->back() mein kya fark hai?

route() specific named route pe redirect karta hai. back() previous page pe redirect karta hai. Mostly store/update ke baad ->route() use karte hain.

Q23: @csrf kyun zaruri hai?

CSRF = Cross-Site Request Forgery. @csrf ek hidden token add karta hai form mein. Laravel verify karta hai ki request usi site se aayi hai. Bina @csrf ke Laravel form submit reject karta hai (419 error).

Q24: number_format() kaise use kiya?

```
number_format($invoice->total, 2)
```

2 decimal places ke saath format karta hai. Rs. 25000 -> 25,000.00.

Q25: Validation kaise kiya?

```
$request->validate(['invoice_date' => 'date|nullable', 'tax' => 'numeric|nullable']);
```

Automatic validation — agar fail ho toh error ke saath form page pe redirect.

Q26: InvoiceItem mein \$table = 'items' kyun likhna pada?

Laravel by default model name se table name guess karta hai. InvoiceItem -> invoice_items. Lekin actual table naam 'items' tha. \$table property se override kiya.

Q27: ?. ?? ternary operator explain karo.

```
$a ?? $b // $a null hai to $b use karo  
$a ? $b : $c // $a true hai to $b, warna $c  
$a ?: $b // $a falsy hai to $b
```

Q28: Subscription system ka concept explain karo.

Free plan mein monthly 5 invoice limit. CheckSubscription middleware invoice create route pe laga. Monthly count check karta hai. 5+ ho to subscription page pe redirect. Pro plan (Rs. 999) Stripe se purchase karo, unlimited invoices.

Q29: Client portal mein security kaise ensure ki?

ClientMiddleware ensure karta hai sirf 'client' role ke users portal access kar sakein. ClientPortalController mein \$invoice->client_id !== auth()->user()->client_id check karta hai — client doosre ka invoice nahi dekh sakta, abort(403) deta hai.

Q30: Is project ko aur improve karne ke liye kya add karoge?

Search and filter functionality. Pagination on invoice/client lists. Password reset link for clients (currently hardcoded client123). Recurring invoice automation. Multi-currency support. AI-based item description generator using OpenAI API.

